

— 4교시 정리

데이터 탐색과 정리

이 시간에 쓴 코드와 실행 결과

4.1

중심 — 평균과 중앙값

형식

`df['열'].mean()` → 평균
`df['열'].median()` → 중앙값

예제

```
print(round(df['Sales'].mean(), 2))  
print(round(df['Sales'].median(), 2))
```

실행 결과

```
239.35  
53.72
```

평균이 중앙값의 **4배가 넘는다**. 큰 주문 몇 건이 평균을 끌어올린 것이다.

`round(값, 자릿수)` 로 소수점을 정리한다. `Series` 에는 `.round(1)` 도 쓸 수 있다.

4.2

퍼진 정도와 사분위수

코드	기능
<code>.std()</code>	표준편차
<code>.quantile(0.25)</code>	25% 지점 (Q1)
<code>.quantile(0.75)</code>	75% 지점 (Q3)

예제

```
print(round(df['Sales'].std(), 2))
print(round(df['Sales'].quantile(0.25), 2))
print(round(df['Sales'].quantile(0.75), 2))
```

실행 결과

1206.96
17.12
209.50

표준편차 1,207 이 평균 239 보다 크다. 평균 하나로 대표할 수 없는 분포라는 뜻이다.

4.3

한 번에 요약

예제

```
df[['Sales', 'Quantity', 'Profit']].describe()
```

실행 결과

	Sales	Quantity	Profit
count	10239.00	10239.00	10239.00
mean	239.35	3.78	28.61
std	1206.96	2.24	232.30
min	0.00	-7.00	-6599.98
25%	17.12	2.00	1.76
50%	53.72	3.00	8.69
75%	209.50	5.00	29.32
max	104835.00	14.00	8399.98

`describe()` 하나로 개수 · 평균 · 표준편차 · 최솟값 · 사분위수 · 최댓값을 모두 볼 수 있다.

4.4

큰 값 · 작은 값 찾기

형식

df.nlargest(N, '열')

→ 큰 값 N개

df.nsmallest(N, '열')

→ 작은 값 N개

예제

df.nlargest(3, 'Sales')[['Sub-Category', 'Sales', 'Quantity', 'Discount', 'Profit']]

실행 결과

	Sub-Category	Sales	Quantity	Discount	Profit
7547	Tables	104835.00	5	0.4	-69.8900
227	Machines	22638.48	6	0.5	-1811.0784
5920	Copiers	17499.95	5	0.0	8399.9760

매출 상위 3건 중 **둘은 적자**다. 할인율을 함께 보면 이유가 보인다. 이상치를 판단할 때는 **여러 열을 함께** 본다.

4.5

비율 구하기

조건식은 참/거짓의 Series 를 만든다. 여기에 `.mean()` 을 쓰면 **참인 비율**이 나온다.

예제

```
print((df['Profit'] < 0).head(3).tolist())  
print(round((df['Profit'] < 0).mean() * 100, 1))
```

실행 결과

```
[False, True, False]
```

```
18.6
```

주문의 **18.6%가 적자**다. 참은 1, 거짓은 0 으로 계산되므로 평균이 곧 비율이 된다.

4.6

분포 그리기

형식

<code>df['열'].plot(kind='hist', bins=50)</code>	→ 히스토그램
<code>df.boxplot(column='값열', by='그룹열')</code>	→ 그룹별 상자그림

예제

```
import matplotlib.pyplot as plt

df[df['Sales'] < 1000]['Sales'].plot(kind='hist', bins=50, figsize=(9, 3))
plt.title('Sales distribution (under 1000)')
plt.show()
```

전체를 그리면 **큰 값 몇 개 때문에 아무것도 보이지 않는다**. 범위를 잘라 그린 뒤, 잘라 낸 부분은 따로 확인한다.

4.7

이상치 판단 기준

가운데 50% 범위를 **IQR** 이라 하고, 그 **1.5배**를 넘어가면 이상치로 보는 방법이 널리 쓰인다.

예제

```
q1 = df['Sales'].quantile(0.25)
q3 = df['Sales'].quantile(0.75)
iqr = q3 - q1

outliers = df[(df['Sales'] < q1 - 1.5 * iqr) | (df['Sales'] > q3 + 1.5 * iqr)]
print(len(outliers))
```

실행 결과

1191

기준에 걸린 1,191건이 **모두 오류인 것은 아니다**. 계산은 “다른 값에 비해 크다”까지만 알려 준다. 지울지 남길지는 판단이다.

한 장 정리

<code>.mean()</code> · <code>.median()</code>	평균 · 중앙값 — 둘을 함께
<code>.std()</code>	표준편차 (퍼진 정도)
<code>.quantile(0.25)</code> · <code>.quantile(0.75)</code>	Q1 · Q3
<code>.describe()</code>	한 번에 요약
<code>.nlargest(N, '열')</code> · <code>.nsmallest()</code>	큰 값 · 작은 값 N개
<code>(조건).mean() * 100</code>	조건에 맞는 비율(%)
<code>.plot(kind='hist', bins=50)</code>	분포 그리기
<code>.boxplot(column=, by=)</code>	그룹별 상자그림